CANNY EDGE DETECTION

Complete Project Codebook, Simulation Results, and Output Images

Design and Simulation-Based Verification of a Modular Canny Edge Detection Architecture Using Verilog RTL

Python reference | Verilog RTL | QuestaSim 10.7c | Six 32 x 32 grayscale images

Prepared from the project scripts and the simulation/metrics output shared in this conversation.

Document contents

- 1. Project objective and workflow
- 2. Files and folder structure
- 3. Algorithm and numeric conventions
- 4. How to run the project in Google Colab and QuestaSim
- 5. Simulation results and quantitative evaluation
- 6. Output figures and six-image results gallery
- 7. Important limitations and IEEE paper readiness
- Appendix A. Six-image preprocessing Python code
- Appendix B. OpenCV Canny reference code
- Appendix C. Complete integer Python golden reference
- Appendix D. Verilog RTL top module
- Appendix E. Verilog testbench
- Appendix F. QuestaSim run script
- Appendix G. Metrics, PNG export, and results JSON/CSV code

1. Project objective and workflow

The goal is to implement a frame-based Canny edge detector in Verilog RTL, simulate it in QuestaSim, and compare its output with an integer Python golden reference. OpenCV Canny is used as an additional software reference for edge-quality evaluation.

All six input images were converted to 8-bit grayscale and resized to 32 x 32 pixels. Each frame therefore contains 1,024 pixels, and six images contain 6,144 pixels in total.

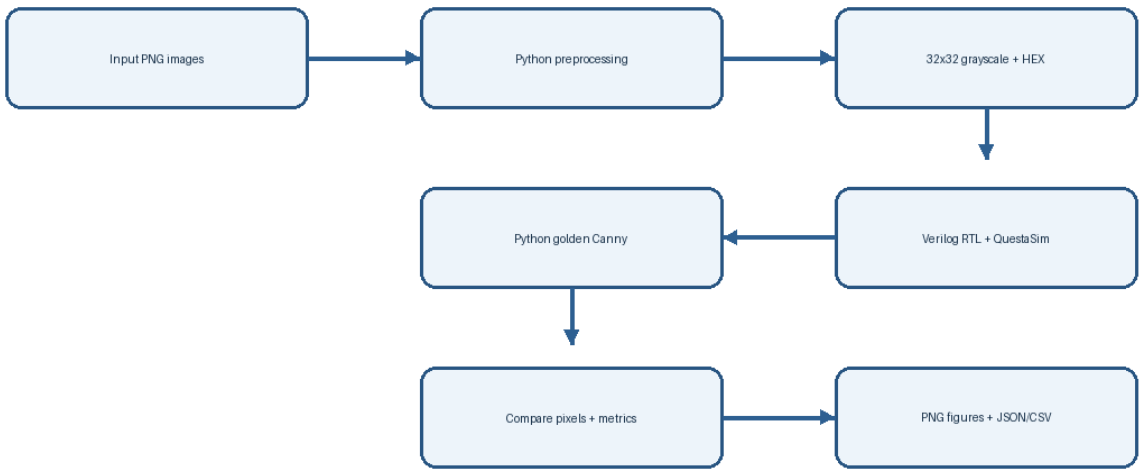


Figure 1. Project data flow from the six input images to Python/RTL comparison outputs.

2. Files and folder structure

File / directory	Purpose
image_to_hex_all.py	Reads l1.jpg-l5.jpg and l6.png; creates 32x32 grayscale PNG and input HEX files.
opencv_canny_reference.py	Generates OpenCV Canny reference edge maps and a comparison image.
python_reference.py	Integer Canny golden model with intermediate stage arrays, PNGs, and

File / directory	Purpose
	reference HEX outputs.
canny_top.v	Frame-based Verilog Canny implementation.
tb_canny_top.v	Loads six test images and Python references; writes six RTL output HEX files and prints mismatch counts.
run_questa.do	Compile, simulate, add waveform signals, log, and save waveform layout.
calculate_metrics_and_export.py	Uploads RTL HEX files, computes P/R/F1 and exact agreement, reconstructs PNGs and writes JSON/CSV/ZIP.

Recommended working directory in Google Colab: /content/canny_6_outputs. The QuestaSim run used C:/questasim64_10.7c/examples as the working directory, with the input and reference HEX files located there.

3. Algorithm and numeric conventions

- Input image: 8-bit grayscale, 32 x 32 pixels.
- Gaussian smoothing: 3 x 3 kernel [[1,2,1],[2,4,2],[1,2,1]] / 16, with round-to-nearest by adding 8 before right-shifting by 4.
- Border treatment: replicate padding for Gaussian and Sobel stages; zero-valued out-of-frame neighbours for NMS.
- Gradient magnitude: L1 approximation M = |Gx| + |Gy|, maximum representable value 2040 for 8-bit Sobel input.
- Gradient direction: four bins using the 106/256 approximation to tan(22.5 degrees).
- NMS tie handling: keep centre if centre >= first neighbour and centre > second neighbour.
- Thresholds: high=120, low=40; class values are 0=none, 1=weak, 2=strong.
- Hysteresis: full iterative 8-connected weak-pixel promotion until no new weak pixels become strong.
- Final output: binary edge map (0x00 for non-edge and 0xFF for strong edge).

4. How to run the project

4.1 Google Colab steps

- Upload the six source images as /content/I1.jpg, /content/I2.jpg, /content/I3.jpg, /content/I4.jpg, /content/I5.jpg, and /content/I6.png.
- Run image_to_hex_all.py to create six grayscale PNGs and six input HEX files.
- Run python_reference.py to create the integer golden maps, stage PNGs, reference HEX files, NPZ stage arrays, and reference summary.
- Optionally run opencv_canny_reference.py to create the independent OpenCV Canny outputs.
- After QuestaSim creates rtl_image1_output.hex through rtl_image6_output.hex, run calculate_metrics_and_export.py and select those six files in the upload dialog.
- Download results.json, results.csv, rtl_python_opencv_comparison.png, and canny_complete_results.zip from the output folder.

4.2 QuestaSim steps

Copy the twelve vectors (image1.hex to image6.hex and image1_reference.hex to image6_reference.hex) into the QuestaSim working directory. Keep canny_top.v and tb_canny_top.v in the same directory. Then execute:

```
vlib work
vlog canny_top.v
vlog tb_canny_top.v
vsim -wlf canny_full_waveform.wlf -voptargs=+acc work.tb_canny_top
run -all
```

The successful run reported zero pixel mismatches for all six frames. The user also verified creation of canny_full_waveform.wlf and canny_waveform.do in C:/questasim64_10.7c/examples/cadproject/Canny_results/. A waveform screenshot file was not confirmed in the provided materials.

5. Simulation results and quantitative evaluation

The following values are from the successful QuestaSim log and the Python metrics output shared in the conversation. The initial simulation attempt with missing HEX files is excluded because its mismatch values were invalid.

Image	RTL edges	OpenCV edges	P	R	F1	Full-hyst F1	Max |Gx|+|Gy|	RTL/Python agreement
1	301	298	0.9967	1.0000	0.9983	0.9983	570	100.00%
2	173	172	1.0000	1.0000	1.0000	1.0000	740	100.00%
3	326	320	0.9969	1.0000	0.9985	0.9985	646	100.00%
4	111	111	1.0000	1.0000	1.0000	1.0000	510	100.00%
5	295	291	1.0000	1.0000	1.0000	1.0000	356	100.00%
6	268	261	0.9813	1.0000	0.9906	0.9906	756	100.00%

Aggregate metrics

Metric	Measured result
Images tested	6
Pixels compared	6,144
Total RTL edge pixels	1474
Total Python full-hysteresis edge pixels	1474
Total OpenCV edge pixels	1453
Mean precision vs OpenCV	0.9958
Mean recall vs OpenCV	1.0000
Mean F1 vs OpenCV	0.9979
Mean full-hysteresis F1 vs OpenCV	0.9979
Exact RTL/Python agreement	100%
RTL/Python mismatches	0
Measured maximum gradient range	356-756

P/R/F1 use symmetric dilation-based matching with a 1-pixel tolerance; this is not one-to-one edge assignment. Pixel agreement is an exact binary pixel comparison. Thus high F1 and less-than-100% exact OpenCV agreement are compatible. The RTL matches the integer Python full-hysteresis reference exactly on these six tested images, so the F1 values of those two maps against OpenCV are the same.

How to interpret the results

- Zero mismatches over 6,144 pixels demonstrates bit-exact agreement with the current Python reference for these six test images.
- The OpenCV comparison is an additional software baseline; differences are possible because implementations can differ in numerical and edge-linking details.
- Maximum gradient magnitudes were measured per image from the Gaussian-smoothed inputs before NMS. The observed range is 356 to 756; these values are not configurable target values.
- No FPGA synthesis, area, power, maximum clock frequency, or hardware throughput results were measured. Do not claim them in the paper.

6. Output images

The following pages include the actual 32 x 32 image results enlarged with nearest-neighbour scaling for viewing. Each row compares the source grayscale image, Python full-hysteresis golden map, reconstructed Verilog RTL output, and OpenCV Canny map.

Figure 2. Images 1-3: input, Python reference, Verilog RTL, and OpenCV Canny.

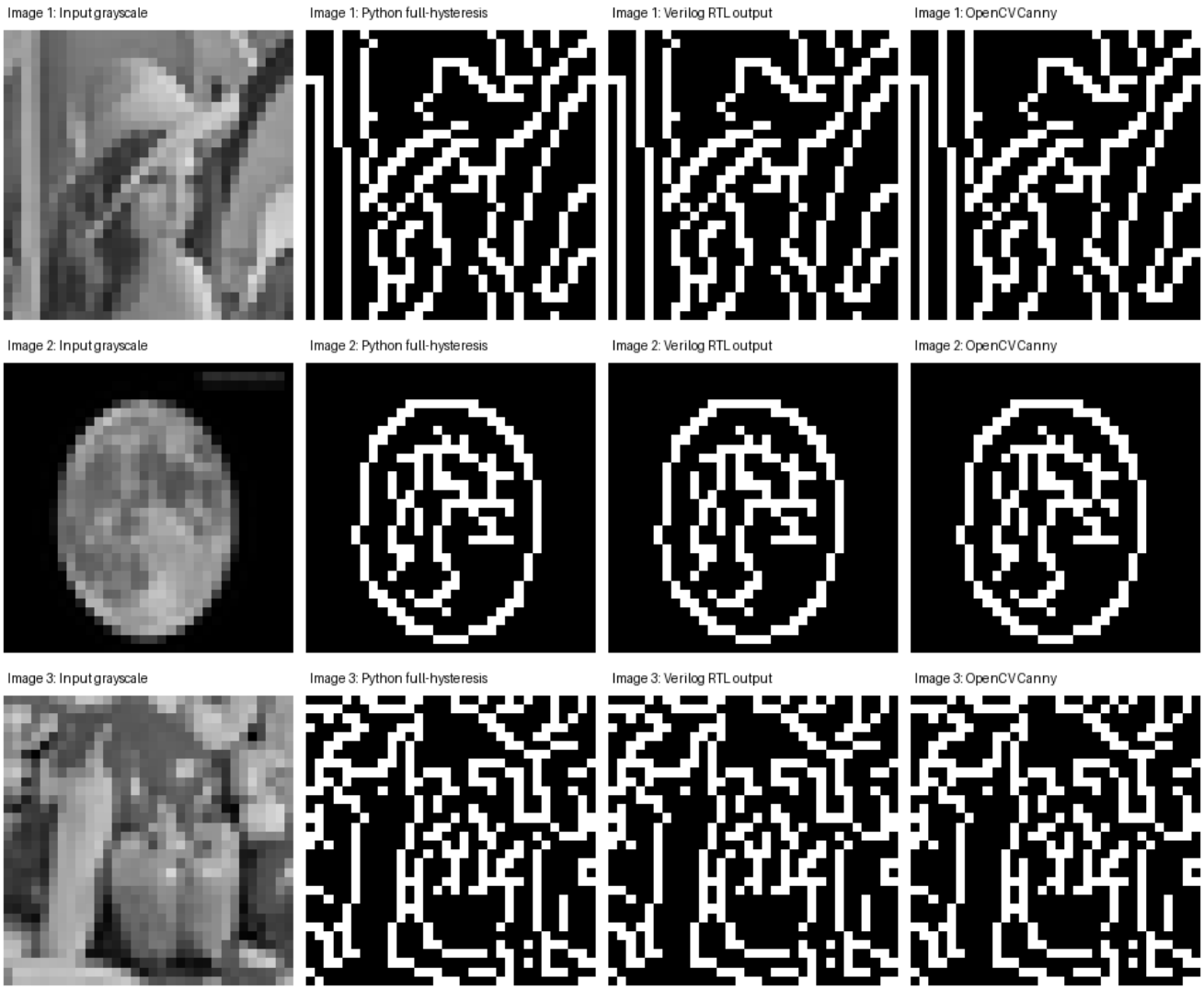


Figure 2. Images 1-3: input, Python reference, Verilog RTL, and OpenCV Canny.

Figure 3. Images 4-6: input, Python reference, Verilog RTL, and OpenCV Canny.

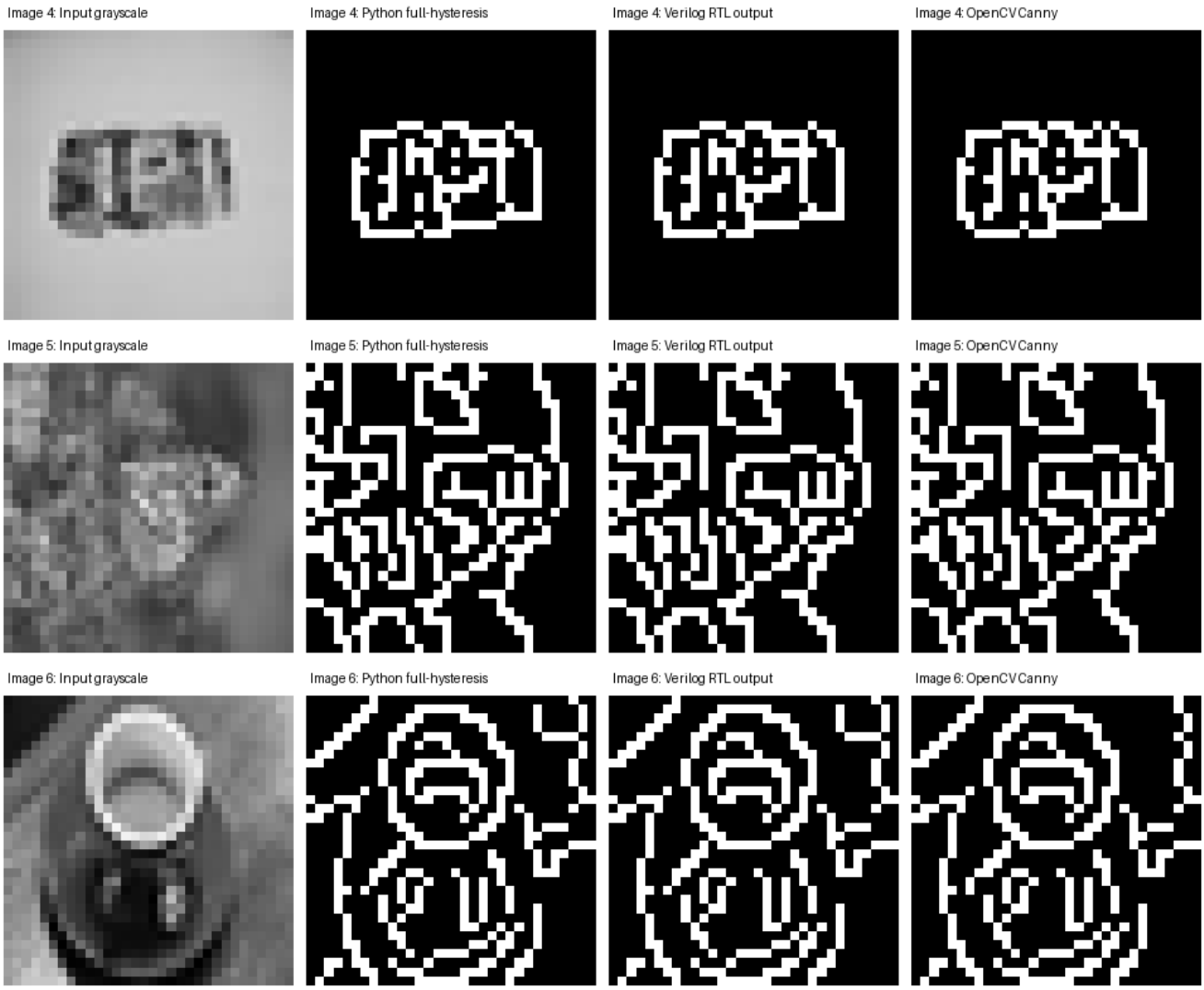


Figure 3. Images 4-6: input, Python reference, Verilog RTL, and OpenCV Canny.

Canny stage-by-stage illustration

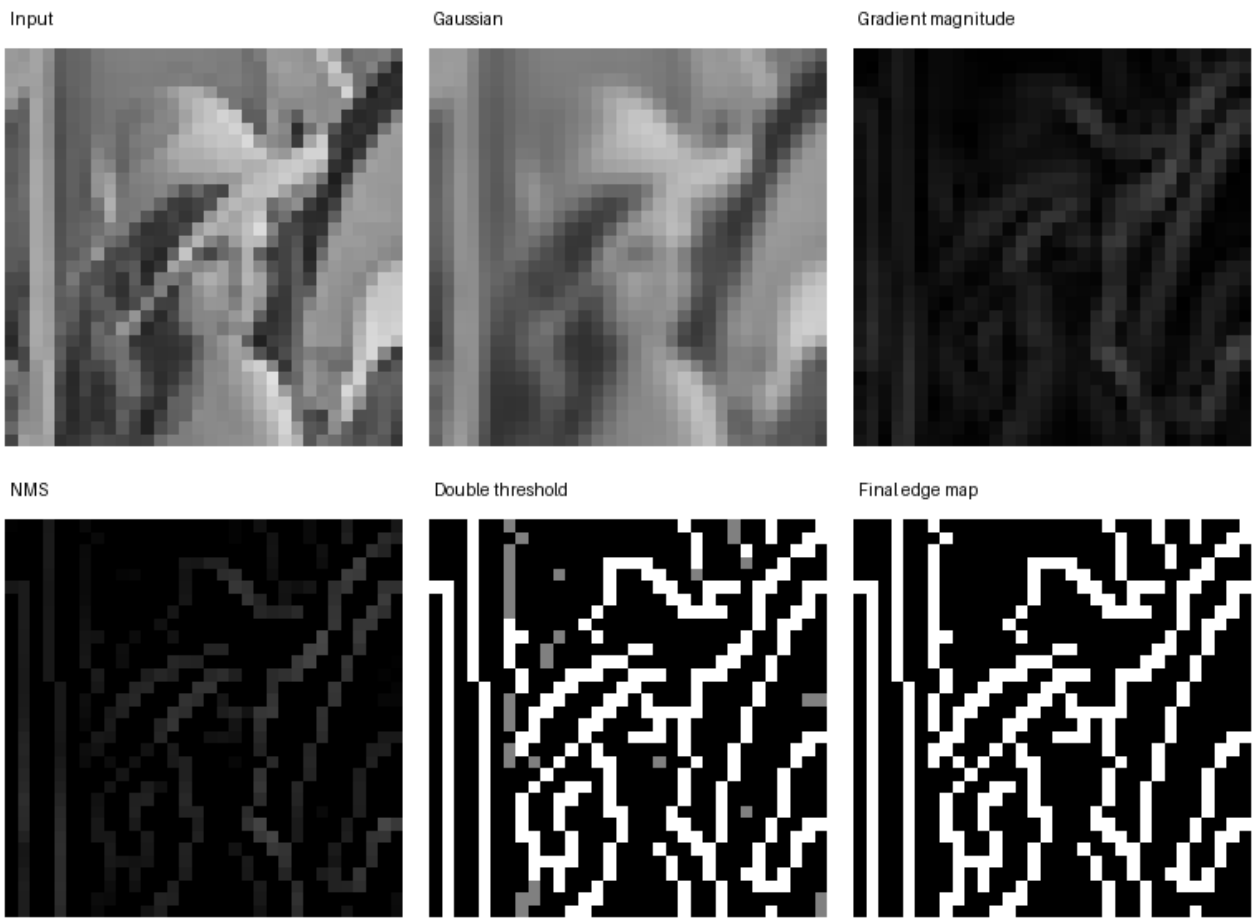


Figure 4. Image 1 example: grayscale input, Gaussian smoothed image, gradient magnitude, NMS, double-threshold classes, and final Python edge map.

7. IEEE paper readiness and limitations

The basic implementation and six-image functional verification are complete. The manuscript is not yet guaranteed ready for IEEE submission. Existing Canny RTL/hardware implementations mean the paper must articulate a specific, defensible contribution and cite related work accurately.

- Current RTL organization: the processing stages are implemented as functions/state-machine operations in canny_top.v. Although the stages are logically separated, the RTL is not yet a set of independent stand-alone modules. Either refactor into separate modules or adjust the title/claims to describe the delivered design accurately.
- Expand verification beyond six 32 x 32 natural-image samples: constant images, single-pixel impulse, horizontal/vertical/diagonal step edges, checkerboard, odd image dimensions if supported, random noise, reset behavior, and boundary cases.
- Document exactly how the independent OpenCV Canny output is generated and how tolerance-based P/R/F1 is calculated. The used symmetric-dilation metric should be stated clearly.
- The provided waveform .wlf and .do files were verified as existing in the Windows working folder; no actual waveform screenshot was provided in the saved artifacts.
- No FPGA hardware, synthesis estimate, timing analysis, power analysis, or area measurement has been performed. Mention this as a limitation unless those experiments are added.
- Before submission, compare against related prior work and confirm that the chosen IEEE conference accepts simulation-based design papers.

Simulation log excerpt

```
========== IMAGE 1 ==========
Expected edge pixels : 301
RTL edge pixels      : 301
Pixel mismatches     : 0
RESULT: PASS - exact pixel match

========== IMAGE 2 ==========
Expected edge pixels : 173
RTL edge pixels      : 173
Pixel mismatches     : 0
RESULT: PASS - exact pixel match

========== IMAGE 3 ==========
Expected edge pixels : 326
RTL edge pixels      : 326
Pixel mismatches     : 0
RESULT: PASS - exact pixel match

========== IMAGE 4 ==========
Expected edge pixels : 111
RTL edge pixels      : 111
Pixel mismatches     : 0
RESULT: PASS - exact pixel match

========== IMAGE 5 ==========
Expected edge pixels : 295
RTL edge pixels      : 295
Pixel mismatches     : 0
RESULT: PASS - exact pixel match

========== IMAGE 6 ==========
Expected edge pixels : 268
RTL edge pixels      : 268
Pixel mismatches     : 0
RESULT: PASS - exact pixel match

All six image simulations completed.
```

Appendix Code: image_to_hex_all.py

Language: Python | Source lines: 38

Run in Google Colab after uploading l1.jpg through l5.jpg and l6.png to /content.

```
"""Convert six uploaded images into 32x32 grayscale PNG and HEX inputs.

Google Colab assumptions: source images are /content/l1.jpg ... /content/l6.png.
Each HEX file stores one 8-bit pixel per line, row-major, two-digit hexadecimal.
"""
from pathlib import Path
from PIL import Image
import numpy as np
import zipfile

ROOT = Path('/content')
INPUTS = [ROOT / f'l{i}.jpg' for i in range(1, 6)] + [ROOT / 'l6.png']
OUTPUT = ROOT / 'canny_6_outputs'
PNG_DIR = OUTPUT / 'png'
HEX_DIR = OUTPUT / 'hex'
PNG_DIR.mkdir(parents=True, exist_ok=True)
HEX_DIR.mkdir(parents=True, exist_ok=True)

for i, src in enumerate(INPUTS, start=1):
    if not src.exists():
        raise FileNotFoundError(f'Input image not found: {src}')
    with Image.open(src) as im:
        gray = im.convert('L').resize((32, 32), Image.Resampling.LANCZOS)
    png_path = PNG_DIR / f'image{i}_32x32.png'
    hex_path = HEX_DIR / f'image{i}.hex'
    gray.save(png_path, format='PNG')
    pixels = np.asarray(gray, dtype=np.uint8)
    with hex_path.open('w', encoding='utf-8') as f:
        for pixel in pixels.flatten():
            f.write(f'{int(pixel):02X}\n')
    print(f'Image {i}: {src.name} -> {png_path.name}, {hex_path.name}; shape={pixels.shape}; pixels={pixels.size}')

zip_path = ROOT / 'canny_6_images_outputs.zip'
with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as zf:
    for folder in (PNG_DIR, HEX_DIR):
        for path in folder.glob('*'):
            zf.write(path, arcname=str(path.relative_to(OUTPUT)))
print(f'Created {zip_path}')
```

Appendix Code: opencv_canny_reference.py

Language: Python | Source lines: 50

Generates an independent OpenCV reference and preview montage.

```
"""Generate OpenCV Canny reference maps for the six 32x32 input images.

This is an independent software baseline for edge-quality comparison; it is not
an RTL result and may differ from the integer Canny reference due to algorithm
implementation details.
"""
from pathlib import Path
import cv2
import numpy as np
from PIL import Image, ImageDraw
import zipfile

BASE = Path('/content/canny_6_outputs')
INPUT_DIR = BASE / 'png'
OUT_DIR = BASE / 'opencv_reference'
OUT_DIR.mkdir(parents=True, exist_ok=True)
LOW, HIGH = 40, 120
records = []

for i in range(1, 7):
    input_path = INPUT_DIR / f'image{i}_32x32.png'
    if not input_path.exists():
        raise FileNotFoundError(f'Missing input PNG: {input_path}')
    gray = cv2.imread(str(input_path), cv2.IMREAD_GRAYSCALE)
    if gray is None:
        raise ValueError(f'Unable to read {input_path}')
    gaussian = cv2.GaussianBlur(gray, (3, 3), 0, borderType=cv2.BORDER_REPLICATE)
    edges = cv2.Canny(gaussian, LOW, HIGH, apertureSize=3, L2gradient=False)
    out_path = OUT_DIR / f'image{i}_opencv_canny.png'
    Image.fromarray(edges).save(out_path)
    records.append((i, Image.fromarray(gray), Image.fromarray(edges)))
    print(f'Image {i}: OpenCV edge pixels={int(np.count_nonzero(edges))}; saved {out_path}')

thumb = 192
label_h = 24
sheet = Image.new('RGB', (thumb * 2, (thumb + label_h) * 6), 'white')
draw = ImageDraw.Draw(sheet)
for row, (i, original, edges) in enumerate(records):
    y = row * (thumb + label_h)
    draw.text((0, y + 4), f'Image {i} input', fill='black')
    draw.text((thumb, y + 4), f'Image {i} OpenCV Canny', fill='black')
    sheet.paste(original.resize((thumb, thumb), Image.Resampling.NEAREST).convert('RGB'), (0, y + label_h))
    sheet.paste(edges.resize((thumb, thumb), Image.Resampling.NEAREST).convert('RGB'), (thumb, y + label_h))
comparison = OUT_DIR / 'opencv_comparison.png'
sheet.save(comparison)
zip_path = Path('/content/canny_opencv_reference.zip')
with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as zf:
    for p in OUT_DIR.rglob('*'):
        if p.is_file(): zf.write(p, arcname=str(p.relative_to(OUT_DIR)))
print('OpenCV reference generation complete:', zip_path)
```

Appendix Code: python_reference.py

Language: Python | Source lines: 357

This is the saved 14 KB integer reference script used to create Python stage maps and reference HEX files.

```
#!/usr/bin/env python3
"""Bit-accurate, frame-based Canny reference for the companion Verilog RTL.

Algorithm and numeric conventions are deliberately matched to rtl/canny_top.v:
 * 8-bit grayscale input
 * 3x3 Gaussian kernel [1 2 1; 2 4 2; 1 2 1] / 16, rounded by +8 then >>4
 * replicated borders for Gaussian and Sobel
 * Sobel magnitude = abs(Gx) + abs(Gy)
 * direction quantization using 106/256 ~= tan(22.5 degrees)
 * NMS uses zero-valued out-of-frame neighbors and asymmetric tie handling
 * double thresholds classify 0=none, 1=weak, 2=strong
 * full iterative 8-connected hysteresis until no more weak pixels are promoted

Default input directory is the folder created by the user's six-image preprocessing
script in Google Colab: /content/canny_6_outputs/png/image1_32x32.png ... image6_32x32.png.
If those are absent, the script falls back to /content/l1.jpg ... /content/l6.png.
"""

from pathlib import Path
import csv
import zipfile
import numpy as np
from PIL import Image, ImageDraw

# ---------------------------- User settings ---------------------------------
WIDTH = 32
HEIGHT = 32
THRESHOLD_HIGH = 120
THRESHOLD_LOW = 40

COLAB_ROOT = Path('/content')
PROCESSED_INPUT_DIR = COLAB_ROOT / 'canny_6_outputs' / 'png'
RAW_IMAGE_PATHS = [
    COLAB_ROOT / 'l1.jpg',
    COLAB_ROOT / 'l2.jpg',
    COLAB_ROOT / 'l3.jpg',
    COLAB_ROOT / 'l4.jpg',
    COLAB_ROOT / 'l5.jpg',
    COLAB_ROOT / 'l6.png',
]
OUTPUT_ROOT = COLAB_ROOT / 'canny_6_outputs' / 'python_reference'

# ---------------------------- Canny stages ----------------------------------
def read_gray_image(path: Path) -> np.ndarray:
    """Read image as 8-bit grayscale and resize it to WIDTH x HEIGHT."""
    with Image.open(path) as image:
        image = image.convert('L')
        if image.size != (WIDTH, HEIGHT):
            image = image.resize((WIDTH, HEIGHT), Image.Resampling.LANCZOS)
        return np.asarray(image, dtype=np.uint8).copy()


def gaussian_3x3(image: np.ndarray) -> np.ndarray:
    """3x3 integer Gaussian with replicate padding and round-to-nearest."""
    padded = np.pad(image.astype(np.int32), ((1, 1), (1, 1)), mode='edge')
    p0 = padded[:-2, :-2]
    p1 = padded[:-2, 1:-1]
    p2 = padded[:-2, 2:]
    p3 = padded[1:-1, :-2]
    p4 = padded[1:-1, 1:-1]
    p5 = padded[1:-1, 2:]
    p6 = padded[2:, :-2]
    p7 = padded[2:, 1:-1]
    p8 = padded[2:, 2:]

    total = (p0 + 2*p1 + p2 + 2*p3 + 4*p4 + 2*p5
             + p6 + 2*p7 + p8)
    return ((total + 8) >> 4).clip(0, 255).astype(np.uint8)


def sobel_gradient(image: np.ndarray):
    """Compute signed Gx/Gy, L1 magnitude, and four-sector direction."""
    padded = np.pad(image.astype(np.int32), ((1, 1), (1, 1)), mode='edge')
    p0 = padded[:-2, :-2]
    p1 = padded[:-2, 1:-1]
    p2 = padded[:-2, 2:]
    p3 = padded[1:-1, :-2]
    p5 = padded[1:-1, 2:]
```

```
    p6 = padded[2:, :-2]
    p7 = padded[2:, 1:-1]
    p8 = padded[2:, 2:]

    # Same equations used by the companion Verilog sobel_grad module.
    gx = (p2 + 2*p5 + p8) - (p0 + 2*p3 + p6)
    gy = (p6 + 2*p7 + p8) - (p0 + 2*p1 + p2)
    ax = np.abs(gx)
    ay = np.abs(gy)
    magnitude = (ax + ay).astype(np.uint16)  # range fits in 11 bits

    # Sector coding matches sobel_grad.v:
    # 0=W/E, 1=NW/SE, 2=N/S, 3=NE/SW.
    direction = np.ones(gx.shape, dtype=np.uint8)
    horizontal = (256 * ay) < (106 * ax)
    vertical = (256 * ax) < (106 * ay)
    direction[horizontal] = 0
    direction[vertical] = 2

    diagonal = ~(horizontal | vertical)
    same_sign = ((gx >= 0) & (gy >= 0)) | ((gx < 0) & (gy < 0))
    direction[diagonal & same_sign] = 1
    direction[diagonal & ~same_sign] = 3

    return gx.astype(np.int16), gy.astype(np.int16), magnitude, direction


def non_maximum_suppression(magnitude: np.ndarray,
                            direction: np.ndarray) -> np.ndarray:
    """NMS with zero padding and the RTL's asymmetric tie condition."""
    mag = magnitude.astype(np.uint16)
    padded = np.pad(mag, ((1, 1), (1, 1)), mode='constant', constant_values=0)

    west = padded[1:-1, :-2]
    east = padded[1:-1, 2:]
    nw = padded[:-2, :-2]
    se = padded[2:, 2:]
    north = padded[:-2, 1:-1]
    south = padded[2:, 1:-1]
    ne = padded[:-2, 2:]
    sw = padded[2:, :-2]

    first = np.zeros(mag.shape, dtype=np.uint16)
    second = np.zeros(mag.shape, dtype=np.uint16)

    mask = direction == 0
    first[mask], second[mask] = west[mask], east[mask]
    mask = direction == 1
    first[mask], second[mask] = nw[mask], se[mask]
    mask = direction == 2
    first[mask], second[mask] = north[mask], south[mask]
    mask = direction == 3
    first[mask], second[mask] = ne[mask], sw[mask]

    keep = (mag >= first) & (mag > second)
    return np.where(keep, mag, 0).astype(np.uint16)


def double_threshold(nms: np.ndarray, high: int, low: int) -> np.ndarray:
    """Return classes: 0=non-edge, 1=weak, 2=strong."""
    if not (0 <= low <= high <= 2047):
        raise ValueError('Thresholds must satisfy 0 <= low <= high <= 2047.')
    classes = np.zeros(nms.shape, dtype=np.uint8)
    classes[nms >= low] = 1
    classes[nms >= high] = 2
    return classes


def hysteresis_full(classes: np.ndarray) -> np.ndarray:
    """Promote weak pixels connected to strong pixels using 8-connectivity.

    This iterates until convergence, matching the repeated-pass controller in
    canny_top.v. The array returned contains class values 0, 1, and 2.
    """
    result = classes.copy().astype(np.uint8)
    height, width = result.shape

    # Each pass uses the strong-pixel map from the start of the pass, equivalent
    # to synchronous neighbor propagation. Repeating passes reaches the same
    # connected-component closure as the RTL's repeated scans.
    while True:
        strong = (result == 2)
        padded = np.pad(strong, ((1, 1), (1, 1)), mode='constant', constant_values=False)
        connected = (
            padded[:-2, :-2] | padded[:-2, 1:-1] | padded[:-2, 2:] |
            padded[1:-1, :-2] |                    padded[1:-1, 2:] |
```

```
            padded[2:, :-2] | padded[2:, 1:-1] | padded[2:, 2:]
        )
        promote = (result == 1) & connected
        if not np.any(promote):
            break
        result[promote] = 2

    return result


def run_canny_reference(image: np.ndarray, high: int = THRESHOLD_HIGH,
                        low: int = THRESHOLD_LOW) -> dict:
    """Run the complete integer Canny reference and return every stage."""
    if image.shape != (HEIGHT, WIDTH):
        raise ValueError(f'Expected {(HEIGHT, WIDTH)}, got {image.shape}.')
    if not (0 <= low <= high <= 2047):
        raise ValueError('Require 0 <= threshold_low <= threshold_high <= 2047.')

    gaussian = gaussian_3x3(image)
    gx, gy, magnitude, direction = sobel_gradient(gaussian)
    nms = non_maximum_suppression(magnitude, direction)
    classes = double_threshold(nms, high, low)
    linked_classes = hysteresis_full(classes)
    edges = np.where(linked_classes == 2, 255, 0).astype(np.uint8)

    return {
        'input': image,
        'gaussian': gaussian,
        'gx': gx,
        'gy': gy,
        'magnitude': magnitude,
        'direction': direction,
        'nms': nms,
        'classes': classes,
        'linked_classes': linked_classes,
        'edges': edges,
    }

# ---------------------------- Saving helpers --------------------------------
def save_hex_8bit(path: Path, values: np.ndarray) -> None:
    """Save uint8-style values as one two-digit hexadecimal value per line."""
    with path.open('w', encoding='utf-8') as f:
        for value in values.flatten():
            f.write(f'{int(value) & 0xFF:02X}\n')


def display_scaled(values: np.ndarray, maximum: int) -> Image.Image:
    """Scale a numeric stage to an 8-bit preview image."""
    scaled = np.rint(np.clip(values.astype(np.float64) / max(maximum, 1), 0, 1) * 255)
    return Image.fromarray(scaled.astype(np.uint8), mode='L')


def save_stage_images(stages: dict, prefix: str, png_dir: Path) -> None:
    """Save visual previews for the useful algorithm stages."""
    Image.fromarray(stages['input']).save(png_dir / f'{prefix}_input.png')
    Image.fromarray(stages['gaussian']).save(png_dir / f'{prefix}_gaussian.png')
    display_scaled(stages['magnitude'], 2040).save(png_dir / f'{prefix}_gradient_magnitude.png')
    display_scaled(stages['nms'], 2040).save(png_dir / f'{prefix}_nms.png')
    class_preview = np.zeros(stages['classes'].shape, dtype=np.uint8)
    class_preview[stages['classes'] == 1] = 128
    class_preview[stages['classes'] == 2] = 255
    Image.fromarray(class_preview).save(png_dir / f'{prefix}_double_threshold.png')
    Image.fromarray(stages['edges']).save(png_dir / f'{prefix}_reference_edges.png')


def make_comparison_sheet(records: list, output_path: Path) -> None:
    """Create a six-row input/final-output comparison sheet."""
    thumb_w, thumb_h = 192, 192
    label_h = 26
    margin = 12
    col_gap = 18
    row_gap = 18
    canvas_w = margin * 2 + 4 * thumb_w + 3 * col_gap
    canvas_h = margin * 2 + 6 * (thumb_h + label_h) + 5 * row_gap
    sheet = Image.new('RGB', (canvas_w, canvas_h), 'white')
    draw = ImageDraw.Draw(sheet)

    for index, record in enumerate(records):
        row = index
        y = margin + row * (thumb_h + label_h + row_gap)
        x1 = margin
        x2 = margin + 2 * thumb_w + col_gap
        draw.text((x1, y), f"{record['name']} - input", fill='black')
        draw.text((x2, y), f"{record['name']} - Canny reference", fill='black')
        in_img = Image.open(record['input_png']).convert('L').resize(
            (thumb_w, thumb_h), Image.Resampling.NEAREST)
```

```
        out_img = Image.open(record['edge_png']).convert('L').resize(
            (thumb_w, thumb_h), Image.Resampling.NEAREST)
        sheet.paste(in_img.convert('RGB'), (x1, y + label_h))
        sheet.paste(out_img.convert('RGB'), (x2, y + label_h))

    sheet.save(output_path)


def main() -> None:
    if not (0 <= THRESHOLD_LOW <= THRESHOLD_HIGH <= 2047):
        raise ValueError('Thresholds must satisfy 0 <= low <= high <= 2047.')

    # Prefer the six already-preprocessed PNGs created by the previous script.
    processed_paths = [
        PROCESSED_INPUT_DIR / f'image{i}_32x32.png' for i in range(1, 7)
    ]
    if all(path.exists() for path in processed_paths):
        input_paths = processed_paths
    else:
        input_paths = RAW_IMAGE_PATHS

    missing = [str(path) for path in input_paths if not path.exists()]
    if missing:
        raise FileNotFoundError(
            'Could not find the six expected input images. Checked paths:\n' +
            '\n'.join(missing) +
            '\nRun the image-preprocessing script first or update INPUT paths.'
        )

    png_dir = OUTPUT_ROOT / 'stage_png'
    hex_dir = OUTPUT_ROOT / 'reference_hex'
    arrays_dir = OUTPUT_ROOT / 'stage_arrays'
    for directory in (png_dir, hex_dir, arrays_dir):
        directory.mkdir(parents=True, exist_ok=True)

    records = []
    summary_rows = []
    print(f'Input source: {input_paths[0].parent}')
    print(f'Thresholds: high={THRESHOLD_HIGH}, low={THRESHOLD_LOW}')

    for index, input_path in enumerate(input_paths, start=1):
        prefix = f'image{index}'
        image = read_gray_image(input_path)
        stages = run_canny_reference(image, THRESHOLD_HIGH, THRESHOLD_LOW)

        save_stage_images(stages, prefix, png_dir)
        output_hex = hex_dir / f'{prefix}_reference.hex'
        save_hex_8bit(output_hex, stages['edges'])

        # Save unscaled numeric values as a NumPy archive for exact inspection.
        np.savez_compressed(
            arrays_dir / f'{prefix}_stages.npz',
            input=stages['input'], gaussian=stages['gaussian'],
            gx=stages['gx'], gy=stages['gy'],
            magnitude=stages['magnitude'], direction=stages['direction'],
            nms=stages['nms'], classes=stages['classes'],
            linked_classes=stages['linked_classes'], edges=stages['edges']
        )

        input_png = png_dir / f'{prefix}_input.png'
        edge_png = png_dir / f'{prefix}_reference_edges.png'
        records.append({'name': prefix, 'input_png': input_png, 'edge_png': edge_png})
        summary_rows.append({
            'image': input_path.name,
            'width': WIDTH,
            'height': HEIGHT,
            'threshold_high': THRESHOLD_HIGH,
            'threshold_low': THRESHOLD_LOW,
            'edge_pixels': int(np.count_nonzero(stages['edges'])),
            'strong_pixels_after_hysteresis': int(np.count_nonzero(stages['linked_classes'] == 2)),
            'weak_pixels_remaining': int(np.count_nonzero(stages['linked_classes'] == 1)),
            'reference_hex': str(output_hex),
        })

        print(f'\n{prefix}: {input_path.name}')
        print(f'  Image dimensions: {image.shape}')
        print(f'  Edge pixels: {summary_rows[-1]['edge_pixels']}')
        print(f'  Reference edge PNG: {edge_png}')
        print(f'  Reference HEX: {output_hex}')

    # Write a CSV summary.
    csv_path = OUTPUT_ROOT / 'reference_summary.csv'
    with csv_path.open('w', newline='', encoding='utf-8') as f:
        writer = csv.DictWriter(f, fieldnames=list(summary_rows[0].keys()))
        writer.writeheader()
        writer.writerows(summary_rows)
```

```
    # Make a comparison sheet and a package for download.
    comparison_path = OUTPUT_ROOT / 'six_image_reference_comparison.png'
    make_comparison_sheet(records, comparison_path)

    zip_path = COLAB_ROOT / 'canny_python_reference_outputs.zip'
    with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as zf:
        for path in OUTPUT_ROOT.rglob('*'):
            if path.is_file():
                zf.write(path, arcname=str(path.relative_to(OUTPUT_ROOT)))

    print('\nREFERENCE GENERATION SUCCESSFUL')
    print('Six reference edge maps generated.')
    print('PNG, HEX, NPZ stage arrays, CSV summary, and comparison image saved.')
    print(f'Output folder: {OUTPUT_ROOT}')
    print(f'Download package: {zip_path}')
    print('\nNote: these are Python golden-reference outputs, not Verilog simulation outputs.')


if __name__ == '__main__':
    main()
```

Appendix Code: canny_top.v

Language: Verilog-2001 | Source lines: 321

Frame-based implementation used for the six-image QuestaSim run reported in this codebook.

```
module canny_top #(
    parameter W  = 32,
    parameter H  = 32,
    parameter AW = 10
)(
    input  wire         clk,
    input  wire         rst_n,
    input  wire         start,
    input  wire         pixel_valid,
    input  wire [7:0]    pixel_in,
    input  wire [10:0]   threshold_high,
    input  wire [10:0]   threshold_low,
    output reg          busy,
    output reg          done,
    output reg          output_valid,
    output reg  [AW-1:0] output_addr,
    output reg  [7:0]    output_pixel
);

    localparam N = W * H;
    localparam ST_IDLE       = 4'd0;
    localparam ST_GAUSSIAN   = 4'd1;
    localparam ST_SOBEL      = 4'd2;
    localparam ST_NMS        = 4'd3;
    localparam ST_THRESHOLD  = 4'd4;
    localparam ST_HYSTERESIS = 4'd5;
    localparam ST_OUTPUT     = 4'd6;

    reg [3:0] state;
    integer idx;
    integer pixel_count;
    reg [10:0] high_reg;
    reg [10:0] low_reg;
    reg pass_changed;

    reg [7:0]  input_mem     [0:N-1];
    reg [7:0]  gaussian_mem  [0:N-1];
    reg [10:0] magnitude_mem [0:N-1];
    reg [1:0]  direction_mem [0:N-1];
    reg [10:0] nms_mem       [0:N-1];
    reg [1:0]  class_mem     [0:N-1];

    // Replicate-border access for the input image.
    function [7:0] input_at;
        input integer row;
        input integer col;
        integer rr, cc;
        begin
            rr = row;
            cc = col;
            if (rr < 0)  rr = 0;
            if (rr >= H) rr = H-1;
            if (cc < 0)  cc = 0;
            if (cc >= W) cc = W-1;
            input_at = input_mem[rr*W + cc];
        end
    endfunction

    // Stage 1: 3x3 Gaussian filter, integer kernel divided by 16.
    function [7:0] gaussian_pixel;
        input integer pixel_index;
        integer row, col;
        integer dx, dy, weight;
        integer total;
        begin
            row = pixel_index / W;
            col = pixel_index % W;
            total = 0;
            for (dy = -1; dy <= 1; dy = dy + 1) begin
                for (dx = -1; dx <= 1; dx = dx + 1) begin
                    if ((dx == 0) && (dy == 0))
                        weight = 4;
                    else if ((dx == 0) || (dy == 0))
                        weight = 2;
                    else
                        weight = 1;
                    total = total + input_at(row+dy, col+dx) * weight;
                end
```

```
        end
        gaussian_pixel = (total + 8) >> 4;
    end
endfunction

// Replicate-border access for the Gaussian-stage image.
function [7:0] gaussian_at;
    input integer row;
    input integer col;
    integer rr, cc;
    begin
        rr = row;
        cc = col;
        if (rr < 0)  rr = 0;
        if (rr >= H) rr = H-1;
        if (cc < 0)  cc = 0;
        if (cc >= W) cc = W-1;
        gaussian_at = gaussian_mem[rr*W + cc];
    end
endfunction

// Stage 2: Sobel gradient, L1 magnitude, four-sector direction.
// Packed value = {direction[1:0], magnitude[10:0]}.
function [12:0] sobel_pack;
    input integer pixel_index;
    integer row, col;
    integer p0,p1,p2,p3,p5,p6,p7,p8;
    integer gx, gy, ax, ay, mag, dir;
    begin
        row = pixel_index / W;
        col = pixel_index % W;
        p0 = gaussian_at(row-1, col-1);
        p1 = gaussian_at(row-1, col);
        p2 = gaussian_at(row-1, col+1);
        p3 = gaussian_at(row,   col-1);
        p5 = gaussian_at(row,   col+1);
        p6 = gaussian_at(row+1, col-1);
        p7 = gaussian_at(row+1, col);
        p8 = gaussian_at(row+1, col+1);

        gx = (p2 + 2*p5 + p8) - (p0 + 2*p3 + p6);
        gy = (p6 + 2*p7 + p8) - (p0 + 2*p1 + p2);
        ax = (gx < 0) ? -gx : gx;
        ay = (gy < 0) ? -gy : gy;
        mag = ax + ay;
        dir = 1;
        if (256*ay < 106*ax)
            dir = 0;
        else if (256*ax < 106*ay)
            dir = 2;
        else if (((gx >= 0) && (gy >= 0)) ||
                 ((gx < 0) && (gy < 0)))
            dir = 1;
        else
            dir = 3;
        sobel_pack = {dir[1:0], mag[10:0]};
    end
endfunction

// Zero-valued boundary neighbours for non-maximum suppression.
function [10:0] magnitude_zero;
    input integer row;
    input integer col;
    begin
        if ((row < 0) || (row >= H) ||
            (col < 0) || (col >= W))
            magnitude_zero = 11'd0;
        else
            magnitude_zero = magnitude_mem[row*W + col];
    end
endfunction

// Stage 3: Non-maximum suppression with asymmetric tie handling.
function [10:0] nms_pixel;
    input integer pixel_index;
    integer row, col;
    reg [10:0] centre, first_neighbour, second_neighbour;
    reg [1:0] dir;
    begin
        row = pixel_index / W;
        col = pixel_index % W;
        centre = magnitude_mem[pixel_index];
        dir = direction_mem[pixel_index];
        first_neighbour = 11'd0;
        second_neighbour = 11'd0;
        case (dir)
```

```
            2'd0: begin
                first_neighbour = magnitude_zero(row, col-1);
                second_neighbour = magnitude_zero(row, col+1);
            end
            2'd1: begin
                first_neighbour = magnitude_zero(row-1, col-1);
                second_neighbour = magnitude_zero(row+1, col+1);
            end
            2'd2: begin
                first_neighbour = magnitude_zero(row-1, col);
                second_neighbour = magnitude_zero(row+1, col);
            end
            2'd3: begin
                first_neighbour = magnitude_zero(row-1, col+1);
                second_neighbour = magnitude_zero(row+1, col-1);
            end
        endcase
        if ((centre >= first_neighbour) &&
            (centre > second_neighbour))
            nms_pixel = centre;
        else
            nms_pixel = 11'd0;
    end
endfunction

// Stage 5: Identify any strong pixel in the 8-neighbourhood.
function has_strong_neighbor;
    input integer pixel_index;
    integer row, col;
    integer dx, dy, rr, cc;
    begin
        row = pixel_index / W;
        col = pixel_index % W;
        has_strong_neighbor = 1'b0;
        for (dy = -1; dy <= 1; dy = dy + 1) begin
            for (dx = -1; dx <= 1; dx = dx + 1) begin
                rr = row + dy;
                cc = col + dx;
                if (!((dx == 0) && (dy == 0))) begin
                    if ((rr >= 0) && (rr < H) &&
                        (cc >= 0) && (cc < W)) begin
                        if (class_mem[rr*W + cc] == 2'd2)
                            has_strong_neighbor = 1'b1;
                    end
                end
            end
        end
    end
endfunction

// Frame-based controller. Inputs are loaded first, then the stages run.
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state       <= ST_IDLE;
        idx         <= 0;
        pixel_count  <= 0;
        high_reg     <= 11'd120;
        low_reg      <= 11'd40;
        pass_changed <= 1'b0;
        busy        <= 1'b0;
        done        <= 1'b0;
        output_valid <= 1'b0;
        output_addr  <= {AW{1'b0}};
        output_pixel <= 8'd0;
    end else begin
        output_valid <= 1'b0;
        done <= 1'b0;
        case (state)
            ST_IDLE: begin
                busy <= 1'b0;
                if (pixel_valid && (pixel_count < N)) begin
                    input_mem[pixel_count] <= pixel_in;
                    pixel_count <= pixel_count + 1;
                end
                if (start && (pixel_count == N)) begin
                    high_reg <= threshold_high;
                    low_reg <= threshold_low;
                    idx <= 0;
                    pass_changed <= 1'b0;
                    busy <= 1'b1;
                    state <= ST_GAUSSIAN;
                end
            end
            ST_GAUSSIAN: begin
                gaussian_mem[idx] <= gaussian_pixel(idx);
                if (idx == N-1) begin
```

```
                idx <= 0;
                state <= ST_SOBEL;
            end else idx <= idx + 1;
        end
        ST_SOBEL: begin
            {direction_mem[idx], magnitude_mem[idx]} <= sobel_pack(idx);
            if (idx == N-1) begin
                idx <= 0;
                state <= ST_NMS;
            end else idx <= idx + 1;
        end
        ST_NMS: begin
            nms_mem[idx] <= nms_pixel(idx);
            if (idx == N-1) begin
                idx <= 0;
                state <= ST_THRESHOLD;
            end else idx <= idx + 1;
        end
        ST_THRESHOLD: begin
            if (nms_mem[idx] >= high_reg)
                class_mem[idx] <= 2'd2;
            else if (nms_mem[idx] >= low_reg)
                class_mem[idx] <= 2'd1;
            else
                class_mem[idx] <= 2'd0;
            if (idx == N-1) begin
                idx <= 0;
                pass_changed <= 1'b0;
                state <= ST_HYSTERESIS;
            end else idx <= idx + 1;
        end
        ST_HYSTERESIS: begin
            if ((class_mem[idx] == 2'd1) &&
                has_strong_neighbor(idx)) begin
                class_mem[idx] <= 2'd2;
                pass_changed <= 1'b1;
            end
            if (idx == N-1) begin
                if (pass_changed ||
                    ((class_mem[idx] == 2'd1) &&
                     has_strong_neighbor(idx))) begin
                    idx <= 0;
                    pass_changed <= 1'b0;
                    state <= ST_HYSTERESIS;
                end else begin
                    idx <= 0;
                    state <= ST_OUTPUT;
                end
            end else idx <= idx + 1;
        end
        ST_OUTPUT: begin
            busy <= 1'b1;
            output_valid <= 1'b1;
            output_addr <= idx;
            output_pixel <= (class_mem[idx] == 2'd2) ? 8'hFF : 8'h00;
            if (idx == N-1) begin
                idx <= 0;
                pixel_count <= 0;
                busy <= 1'b0;
                done <= 1'b1;
                state <= ST_IDLE;
            end else idx <= idx + 1;
        end
        default: begin
            state <= ST_IDLE;
            busy <= 1'b0;
        end
      endcase
    end
  end
endmodule
```

Appendix Code: tb_canny_top.v

Language: Verilog-2001 | Source lines: 175

Reads six input frames and six corresponding Python reference edge maps; writes six RTL output HEX files.

```
`timescale 1ns/1ps

module tb_canny_top;
    parameter W = 32;
    parameter H = 32;
    parameter N = W * H;

    reg clk;
    reg rst_n;
    reg start;
    reg pixel_valid;
    reg [7:0] pixel_in;
    reg [10:0] threshold_high;
    reg [10:0] threshold_low;

    wire busy;
    wire done;
    wire output_valid;
    wire [9:0] output_addr;
    wire [7:0] output_pixel;

    reg [7:0] input_data [0:N-1];
    reg [7:0] expected_data [0:N-1];

    integer output_fd;
    integer mismatch_count;
    integer rtl_edge_count;
    integer expected_edge_count;
    integer i;

    canny_top #(.W(W), .H(H), .AW(10)) dut (
        .clk(clk), .rst_n(rst_n), .start(start),
        .pixel_valid(pixel_valid), .pixel_in(pixel_in),
        .threshold_high(threshold_high), .threshold_low(threshold_low),
        .busy(busy), .done(done), .output_valid(output_valid),
        .output_addr(output_addr), .output_pixel(output_pixel)
    );

    initial begin
        clk = 1'b0;
        forever #5 clk = ~clk;
    end

    // Record every output and compare to the loaded Python reference.
    always @(posedge clk) begin
        if (output_valid === 1'b1) begin
            if (output_fd != 0)
                $fdisplay(output_fd, "%02X", output_pixel);
            if (output_pixel !== expected_data[output_addr])
                mismatch_count = mismatch_count + 1;
            if (output_pixel === 8'hFF)
                rtl_edge_count = rtl_edge_count + 1;
        end
    end

    task run_image;
        input integer image_number;
        integer j;
        begin
            case (image_number)
                1: begin
                    $readmemh("image1.hex", input_data);
                    $readmemh("image1_reference.hex", expected_data);
                    output_fd = $fopen("rtl_image1_output.hex", "w");
                end
                2: begin
                    $readmemh("image2.hex", input_data);
                    $readmemh("image2_reference.hex", expected_data);
                    output_fd = $fopen("rtl_image2_output.hex", "w");
                end
                3: begin
                    $readmemh("image3.hex", input_data);
                    $readmemh("image3_reference.hex", expected_data);
                    output_fd = $fopen("rtl_image3_output.hex", "w");
                end
                4: begin
                    $readmemh("image4.hex", input_data);
                    $readmemh("image4_reference.hex", expected_data);
```

```
            output_fd = $fopen("rtl_image4_output.hex", "w");
        end
        5: begin
            $readmemh("image5.hex", input_data);
            $readmemh("image5_reference.hex", expected_data);
            output_fd = $fopen("rtl_image5_output.hex", "w");
        end
        6: begin
            $readmemh("image6.hex", input_data);
            $readmemh("image6_reference.hex", expected_data);
            output_fd = $fopen("rtl_image6_output.hex", "w");
        end
        default: begin
            $display("ERROR: Invalid image number");
            $finish;
        end
    endcase

    if (output_fd == 0) begin
        $display("ERROR: Could not open RTL output file.");
        $finish;
    end

    mismatch_count = 0;
    rtl_edge_count = 0;
    expected_edge_count = 0;
    for (j = 0; j < N; j = j + 1)
        if (expected_data[j] === 8'hFF)
            expected_edge_count = expected_edge_count + 1;

    // Reset the DUT for each image.
    rst_n = 1'b0;
    start = 1'b0;
    pixel_valid = 1'b0;
    pixel_in = 8'd0;
    threshold_high = 11'd120;
    threshold_low = 11'd40;
    repeat (3) @(negedge clk);
    rst_n = 1'b1;

    // Load 1024 grayscale pixels, one per valid clock.
    for (j = 0; j < N; j = j + 1) begin
        @(negedge clk);
        pixel_valid = 1'b1;
        pixel_in = input_data[j];
    end

    @(negedge clk);
    pixel_valid = 1'b0;
    start = 1'b1;
    @(negedge clk);
    start = 1'b0;

    wait (done === 1'b1);
    repeat (2) @(posedge clk);
    #1;
    $fclose(output_fd);
    output_fd = 0;

    $display("");
    $display("========== IMAGE %0d ==========", image_number);
    $display("Expected edge pixels : %0d", expected_edge_count);
    $display("RTL edge pixels      : %0d", rtl_edge_count);
    $display("Pixel mismatches     : %0d", mismatch_count);
    if (mismatch_count == 0)
        $display("RESULT: PASS - exact pixel match");
    else
        $display("RESULT: MISMATCH - investigate RTL/reference");
    @(negedge clk);
  end
endtask

initial begin
  rst_n = 1'b0;
  start = 1'b0;
  pixel_valid = 1'b0;
  pixel_in = 8'd0;
  threshold_high = 11'd120;
  threshold_low = 11'd40;
  output_fd = 0;
  mismatch_count = 0;
  rtl_edge_count = 0;
  expected_edge_count = 0;

  run_image(1);
  run_image(2);
```

```
        run_image(3);
        run_image(4);
        run_image(5);
        run_image(6);

        $display("");
        $display("All six image simulations completed.");
        $display("Check individual mismatch counts above.");
        $finish;
    end
endmodule
```

Appendix Code: run_questa.do

Language: QuestaSim Tcl | Source lines: 26

Run in the directory containing all source and HEX files. If the command write format is unsupported in a different QuestaSim build, use the available Wave menu/export command for that build.

```
# Run from the directory containing canny_top.v, tb_canny_top.v and all HEX files.
# QuestaSim 10.7c
vlib work
vlog canny_top.v
vlog tb_canny_top.v
vsim -wlf canny_full_waveform.wlf -voptargs=+acc work.tb_canny_top
view wave
log -r /*
add wave -divider "Clock and Control"
add wave sim:/tb_canny_top/clk
add wave sim:/tb_canny_top/rst_n
add wave sim:/tb_canny_top/start
add wave sim:/tb_canny_top/pixel_valid
add wave sim:/tb_canny_top/busy
add wave sim:/tb_canny_top/done
add wave -divider "Input and Output"
add wave -radix hexadecimal sim:/tb_canny_top/pixel_in
add wave sim:/tb_canny_top/output_valid
add wave -radix unsigned sim:/tb_canny_top/output_addr
add wave -radix hexadecimal sim:/tb_canny_top/output_pixel
add wave -divider "Canny Processing"
add wave -radix unsigned sim:/tb_canny_top/dut/state
add wave -radix unsigned sim:/tb_canny_top/dut/idx
run -all
wave zoom full
write format wave canny_waveform.do
```

Appendix Code: calculate_metrics_and_export.py

Language: Python / Google Colab | Source lines: 211

Run after the preprocessing and Python reference scripts. It opens an upload dialog for six RTL output HEX files and creates PNG/CSV/JSON/ZIP deliverables.

```
"""Upload QuestaSim HEX outputs; calculate metrics and export output figures/reports.

Google Colab prerequisites:
  /content/canny_6_outputs/png/image1_32x32.png ... image6_32x32.png
  /content/canny_6_outputs/python_reference/reference_hex/image1_reference.hex ...
  /content/canny_6_outputs/python_reference/stage_png/image1_gaussian.png ...

When run in Colab, the script opens an upload widget for the six
rtl_imageN_output.hex files generated by QuestaSim.
"""
from pathlib import Path
import json, csv, re, zipfile
import numpy as np
import cv2
from PIL import Image, ImageDraw
from google.colab import files

BASE = Path('/content/canny_6_outputs')
INPUT_DIR = BASE / 'png'
REFERENCE_DIR = BASE / 'python_reference' / 'reference_hex'
STAGE_DIR = BASE / 'python_reference' / 'stage_png'
OUT_DIR = BASE / 'verilog_results'
OUT_DIR.mkdir(parents=True, exist_ok=True)
W = H = 32
N = W * H
LOW, HIGH, TOLERANCE = 40, 120, 1
RTL_NAMES = [f'rtl_image{i}_output.hex' for i in range(1, 7)]

# Upload if absent. Normalize Colab's duplicate-name suffix, e.g. (1).
if not all((Path('/content') / name).exists() for name in RTL_NAMES):
    print('Select the six QuestaSim output files rtl_image1_output.hex ... rtl_image6_output.hex')
    uploaded = files.upload()
    for actual_name, content in uploaded.items():
        p = Path(actual_name)
        normalized = re.sub(r' \(\d+\)$', '', p.stem) + p.suffix.lower()
        if normalized in RTL_NAMES:
            (Path('/content') / normalized).write_bytes(content)

missing = [name for name in RTL_NAMES if not (Path('/content') / name).exists()]
if missing:
    raise FileNotFoundError('Missing RTL output files: ' + ', '.join(missing))


def read_hex(path):
    values = [int(line.strip(), 16) for line in path.read_text().splitlines() if line.strip()]
    if len(values) != N:
        raise ValueError(f'{path.name}: expected {N} pixels; found {len(values)}')
    arr = np.asarray(values, dtype=np.uint8).reshape(H, W)
    if not np.all((arr == 0) | (arr == 255)):
        raise ValueError(f'{path.name}: expected binary pixels 00/FF')
    return arr > 0


def edge_metrics(predicted, reference, tolerance=1):
    pred, ref = predicted.astype(bool), reference.astype(bool)
    pred_count, ref_count = int(pred.sum()), int(ref.sum())
    kernel = np.ones((2*tolerance + 1, 2*tolerance + 1), np.uint8)
    dil_ref = cv2.dilate(ref.astype(np.uint8), kernel, iterations=1) > 0
    dil_pred = cv2.dilate(pred.astype(np.uint8), kernel, iterations=1) > 0
    matched_pred = int(np.count_nonzero(pred & dil_ref))
    matched_ref = int(np.count_nonzero(ref & dil_pred))
    precision = matched_pred / pred_count if pred_count else 0.0
    recall = matched_ref / ref_count if ref_count else 0.0
    f1 = 2*precision*recall/(precision+recall) if precision+recall else 0.0
    return {'precision': precision, 'recall': recall, 'f1': f1,
            'predicted_edge_pixels': pred_count, 'reference_edge_pixels': ref_count,
            'matched_predicted_pixels': matched_pred, 'matched_reference_pixels': matched_ref}


def exact_agreement(a, b):
    mismatches = int(np.count_nonzero(a != b))
    return mismatches, 100.0 * (N - mismatches) / N

results = {
    'project': 'Canny Edge Detection RTL Verification',
    'configuration': {
        'image_count': 6, 'width': W, 'height': H, 'pixels_per_image': N,
```

```
        'threshold_low': LOW, 'threshold_high': HIGH,
        'gradient_magnitude': 'abs(Gx) + abs(Gy)',
        'edge_tolerance_pixels': TOLERANCE,
        'precision_recall_f1_definition': 'Symmetric dilation-based matching with +/-1-pixel tolerance; not one-to-one matching.',
        'pixel_agreement_definition': 'Exact binary pixel equality; no tolerance.',
        'opencv_reference': 'cv2.Canny on matching Gaussian-stage image; thresholds 40/120, apertureSize=3, L2gradient=False.'
    },
    'per_image': [], 'aggregate': {}
}
csv_rows, gallery_rows = [], []

for i in range(1, 7):
    rtl_path = Path('/content') / f'rtl_image{i}_output.hex'
    ref_path = REFERENCE_DIR / f'image{i}_reference.hex'
    gaussian_path = STAGE_DIR / f'image{i}_gaussian.png'
    input_path = INPUT_DIR / f'image{i}_32x32.png'
    for path in (ref_path, gaussian_path, input_path):
        if not path.exists():
            raise FileNotFoundError(f'Required Python-side file not found: {path}. Run preprocessing and python_reference.py first.')

    rtl = read_hex(rtl_path)
    full = read_hex(ref_path)
    gaussian = cv2.imread(str(gaussian_path), cv2.IMREAD_GRAYSCALE)
    if gaussian is None or gaussian.shape != (H, W):
        raise ValueError(f'Bad Gaussian-stage image: {gaussian_path}')

    opencv_u8 = cv2.Canny(gaussian, LOW, HIGH, apertureSize=3, L2gradient=False)
    opencv = opencv_u8 > 0

    g = gaussian.astype(np.int32)
    p = np.pad(g, ((1, 1), (1, 1)), mode='edge')
    p0, p1, p2 = p[:-2, :-2], p[:-2, 1:-1], p[:-2, 2:]
    p3, p5 = p[1:-1, :-2], p[1:-1, 2:]
    p6, p7, p8 = p[2:, :-2], p[2:, 1:-1], p[2:, 2:]
    gx = (p2 + 2*p5 + p8) - (p0 + 2*p3 + p6)
    gy = (p6 + 2*p7 + p8) - (p0 + 2*p1 + p2)
    max_gradient = int((np.abs(gx) + np.abs(gy)).max())

    rtl_cv = edge_metrics(rtl, opencv, TOLERANCE)
    full_cv = edge_metrics(full, opencv, TOLERANCE)
    py_mismatch, py_agree = exact_agreement(rtl, full)
    cv_mismatch, cv_agree = exact_agreement(rtl, opencv)

    rtl_count, full_count, cv_count = int(rtl.sum()), int(full.sum()), int(opencv.sum())
    rtl_png = OUT_DIR / f'rtl_image{i}_output.png'
    opencv_png = OUT_DIR / f'opencv_image{i}_reference.png'
    Image.fromarray(rtl.astype(np.uint8)*255).save(rtl_png)
    Image.fromarray(opencv_u8).save(opencv_png)

    item = {
        'image_id': i, 'input_file': input_path.name, 'width': W, 'height': H, 'total_pixels': N,
        'threshold_low': LOW, 'threshold_high': HIGH,
        'rtl_edge_pixels': rtl_count, 'full_hysteresis_edge_pixels': full_count,
        'opencv_edge_pixels': cv_count, 'max_gradient_magnitude_after_gaussian_before_nms': max_gradient,
        'rtl_vs_opencv_tolerance_1px': rtl_cv,
        'full_hysteresis_vs_opencv_tolerance_1px': full_cv,
        'exact_pixel_agreement_rtl_vs_python_percent': py_agree,
        'mismatches_rtl_vs_python': py_mismatch,
        'exact_pixel_agreement_rtl_vs_opencv_percent': cv_agree,
        'mismatches_rtl_vs_opencv': cv_mismatch
    }
    results['per_image'].append(item)

    csv_rows.append({
        'image_id': i, 'rtl_edge_pixels': rtl_count, 'python_full_hysteresis_edge_pixels': full_count,
        'opencv_edge_pixels': cv_count, 'P_precision_vs_opencv': rtl_cv['precision'],
        'R_recall_vs_opencv': rtl_cv['recall'], 'F1_vs_opencv': rtl_cv['f1'],
        'full_hysteresis_F1_vs_opencv': full_cv['f1'],
        'pixel_agreement_RTL_vs_Python_percent': py_agree,
        'pixel_agreement_RTL_vs_OpenCV_percent': cv_agree,
        'max_gradient_magnitude': max_gradient, 'RTL_Python_mismatches': py_mismatch,
        'RTL_OpenCV_mismatches': cv_mismatch
    })
    print(f"Image {i}: P={rtl_cv['precision']:.4f}, R={rtl_cv['recall']:.4f}, F1={rtl_cv['f1']:.4f}, "
          f"full-hyst F1={full_cv['f1']:.4f}, RTL edges={rtl_count}, OpenCV edges={cv_count}, "
          f"max gradient={max_gradient}, RTL/Python agreement={py_agree:.2f}%")

    original = Image.open(input_path).convert('L')
    gallery_rows.append((i, original, full, rtl, opencv))

metrics = {
    'mean_precision_vs_opencv': float(np.mean([x['rtl_vs_opencv_tolerance_1px']['precision'] for x in results['per_image']])),
    'mean_recall_vs_opencv': float(np.mean([x['rtl_vs_opencv_tolerance_1px']['recall'] for x in results['per_image']])),
    'mean_F1_vs_opencv': float(np.mean([x['rtl_vs_opencv_tolerance_1px']['f1'] for x in results['per_image']])),
    'mean_full_hysteresis_F1_vs_opencv': float(np.mean([x['full_hysteresis_vs_opencv_tolerance_1px']['f1'] for x in results['per_image']])),
    'mean_exact_RTL_python_pixel_agreement_percent': float(np.mean([x['exact_pixel_agreement_rtl_vs_python_percent'] for x in results['per_image']])),
```

```
    'total_RTL_Python_mismatches': int(sum(x['mismatches_rtl_vs_python'] for x in results['per_image'])),
    'total_RTL_edge_pixels': int(sum(x['rtl_edge_pixels'] for x in results['per_image'])),
    'total_Python_full_hysteresis_edge_pixels': int(sum(x['full_hysteresis_edge_pixels'] for x in results['per_image'])),
    'total_OpenCV_edge_pixels': int(sum(x['opencv_edge_pixels'] for x in results['per_image'])),
    'minimum_max_gradient_magnitude': int(min(x['max_gradient_magnitude_after_gaussian_before_nms'] for x in results['per_image'])),
    'maximum_max_gradient_magnitude': int(max(x['max_gradient_magnitude_after_gaussian_before_nms'] for x in results['per_image']))
}
results['aggregate'] = {'images': 6, 'pixels_compared': 6144, **metrics}

json_path = OUT_DIR / 'results.json'
csv_path = OUT_DIR / 'results.csv'
with json_path.open('w', encoding='utf-8') as f:
    json.dump(results, f, indent=2)
with csv_path.open('w', newline='', encoding='utf-8') as f:
    writer = csv.DictWriter(f, fieldnames=list(csv_rows[0].keys()))
    writer.writeheader(); writer.writerows(csv_rows)

# Comparison sheet: input, Python full-hysteresis, Verilog output, OpenCV Canny.
scale, label_h, row_h = 150, 24, 174
sheet = Image.new('RGB', (scale*4, row_h*6), 'white')
draw = ImageDraw.Draw(sheet)
for row, (i, original, full, rtl, opencv) in enumerate(gallery_rows):
    y = row * row_h
    panels = [original, Image.fromarray(full.astype(np.uint8)*255),
              Image.fromarray(rtl.astype(np.uint8)*255), Image.fromarray(opencv.astype(np.uint8)*255)]
    labels = [f'Image {i}: Input', f'Image {i}: Python', f'Image {i}: RTL', f'Image {i}: OpenCV']
    for col, (panel, label) in enumerate(zip(panels, labels)):
        x = col * scale
        draw.text((x+3, y+4), label, fill='black')
        sheet.paste(panel.convert('RGB').resize((scale, scale), Image.Resampling.NEAREST), (x, y+label_h))
comparison_path = OUT_DIR / 'rtl_python_opencv_comparison.png'
sheet.save(comparison_path)

zip_path = OUT_DIR / 'canny_complete_results.zip'
with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as zf:
    for p in OUT_DIR.rglob('*'):
        if p.is_file() and p != zip_path:
            zf.write(p, arcname=str(p.relative_to(OUT_DIR)))

print('\nMETRICS COMPLETE')
print('JSON:', json_path)
print('CSV:', csv_path)
print('PNG comparison:', comparison_path)
print('ZIP:', zip_path)
print(json.dumps(results['aggregate'], indent=2))
files.download(str(json_path))
files.download(str(csv_path))
files.download(str(zip_path))
```

Final deliverables checklist

Artifact	What it contains / status
image_to_hex_all.py	Preprocessing code included.
opencv_canny_reference.py	OpenCV comparison code included.
python_reference.py	Complete integer Canny golden model included.
canny_top.v	Verilog top-level design used in reported simulation included.
tb_canny_top.v	Six-image testbench included.
run_questa.do	Simulation and waveform commands included.
calculate_metrics_and_export.py	Full metrics and results-export code included.
Six-image result galleries	Input, Python, RTL and OpenCV outputs embedded in this PDF.
results.json / results.csv	Generated by metrics code; original JSON is also included in the supplemental bundle.
Waveform screenshot	Not confirmed in provided files; WLF data and DO layout were reported as saved.

The companion source bundle includes all code files, the original 32x32 input PNG/HEX files, Python stage outputs and reference HEX data, six RTL output HEX files, results.json, and the figures used in this PDF.